a. The two genAi used are Claude and Copilot
b. Both were accessed on the web

The C++ program will prioritize emails for a busy company CEO.

• You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.

• You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.

• You must create functions from scratch. Do not include pre-existing heap modules.

• Test File format:

Command Description

EMAIL <sender category>,

<subject line>,

<date>

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

• Boss – read first

• Subordinate – read next

• Peer – read next

• ImportantPerson – read next

• OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

READ CEO has read email and dealt with it

COUNT Display current count of unread emails

Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

C. Copilot:

```
#include <iostream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

// Return numeric priority for each sender category
int categoryPriority(const string& cat) {
if (cat == "Boss") return 5;
if (cat == "Subordinate") return 4;
if (cat == "Peer") return 3;
if (cat == "ImportantPerson") return 2;
return 1;
}

// Convert MM-DD-YYYY into YYYYMMDD integer
int convertDate(const string& date) {
int mm, dd, yyyy;
char dash;

stringstream ss(date);
ss >> mm >> dash >> dd >> dash >> yyyy;

return yyyy * 10000 + mm * 100 + dd;
}

// Email structure
struct Email {
string sender;
string subject;
string dateStr;
int priority;
int dateValue;
};

// Max Heap class
```

```
class MaxHeap {
private:
vector<Email> heap;

// Compare emails by priority first, then date
bool compareEmails(const Email& a, const Email& b) {
if (a.priority != b.priority)
return a.priority > b.priority;

return a.dateValue > b.dateValue;
}

// Restore heap upward
void heapifyUp(int index) {
while (index > 0) {
int parent = (index - 1) / 2;

if (compareEmails(heap[index], heap[parent])) {
swap(heap[index], heap[parent]);
index = parent;
} else {
break;
}
}
}

// Restore heap downward
void heapifyDown(int index) {
int size = heap.size();

while (true) {
int left = 2 * index + 1;
int right = 2 * index + 2;
int largest = index;

if (left < size &&
compareEmails(heap[left], heap[largest])) {
largest = left;
}

if (right < size &&
compareEmails(heap[right], heap[largest])) {
```

```
largest = right;
}

if (largest != index) {
swap(heap[index], heap[largest]);
index = largest;
} else {
break;
}
}
}

public:
// Insert email
void insertEmail(const Email& e) {
heap.push_back(e);
heapifyUp(heap.size() - 1);
}

// Get highest-priority email
Email* getMax() {
if (heap.empty())
return nullptr;

return &heap[0];
}

// Delete highest-priority email
void deleteMax() {
if (heap.empty())
return;

heap[0] = heap.back();
heap.pop_back();

if (!heap.empty()) {
heapifyDown(0);
}
}

// Return current size
int size() const {
```

```
return heap.size();
}
};

int main() {
MaxHeap inbox;
string line;

while (getline(cin, line)) {

// EMAIL command
if (line.substr(0, 5) == "EMAIL") {

string data = line.substr(6);

string sender, subject, date;
stringstream ss(data);

getline(ss, sender, ',');
getline(ss, subject, ',');
getline(ss, date);

Email e;
e.sender = sender;
e.subject = subject;
e.dateStr = date;
e.priority = categoryPriority(sender);
e.dateValue = convertDate(date);

inbox.insertEmail(e);
}

// COUNT command
else if (line.substr(0, 5) == "COUNT") {
cout << "There are "
<< inbox.size()
<< " emails to read.\n";
}

// NEXT command
else if (line.substr(0, 4) == "NEXT") {
```

```
Email* e = inbox.getMax();

if (!e) {
cout << "No emails to read.\n";
}
else {
cout << "Next email:\n";
cout << "Sender: " << e->sender << '\n';
cout << "Subject: " << e->subject << '\n';
cout << "Date: " << e->dateStr << '\n';
}
}

// READ command
else if (line.substr(0, 4) == "READ") {
inbox.deleteMax();
}
}

return 0;
}
```

e. the program is correct and solves the problem perfectly. The only possible problem would be heap overflow. The worst case time complexity is O(log n). The space complexity is O(n) where n is emails. All functions in the code have clear names and comments explain much of the code.

Claude:

```
/* ============================================================
email_priority.cpp

A priority-queue based Email Inbox system for a busy CEO.
Implemented with a hand-written vector based MaxHeap
(no std::priority_queue or <algorithm> heap functions used).

Commands supported (read one per line from a test file):
EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
NEXT
READ
COUNT

Priority order (highest to lowest):
```

```
Boss > Subordinate > Peer > ImportantPerson > OtherPerson
Ties (same category) are broken by date - newest first.

Compile: g++ -std=c++11 -o email_priority email_priority.cpp
Run: ./email_priority testfile.txt
(or) ./email_priority < testfile.txt
============================================================ */

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <utility>

/* ---------- Email record ---------- */

struct Email {
std::string sender; // category string, e.g. "Boss"
std::string subject;
std::string date; // original MM-DD-YYYY string
int priority; // numeric priority, higher = more important
long dateValue; // YYYYMMDD form, higher = newer
};

/* ---------- MaxHeap (vector based) ---------- */

class MaxHeap {
public:
void insert(const Email &e) {
data_.push_back(e);
heapifyUp(static_cast<int>(data_.size()) - 1);
}

// Removes and returns the max element. Returns false if empty.
bool extractMax(Email &out) {
if (data_.empty()) return false;

out = data_[0];
data_[0] = data_.back();
data_.pop_back();
if (!data_.empty()) {
```

```
heapifyDown(0);
}
return true;
}

// Peek at the max element without removing it. Returns false if empty.
bool peekMax(Email &out) const {
if (data_.empty()) return false;
out = data_[0];
return true;
}

int count() const {
return static_cast<int>(data_.size());
}

private:
std::vector<Email> data_;

// Returns true if a should be HIGHER priority than b
// (i.e. a should sit above/before b in the max-heap).
static bool isHigherPriority(const Email &a, const Email &b) {
if (a.priority != b.priority) {
return a.priority > b.priority;
}
// same category -> newest date wins
return a.dateValue > b.dateValue;
}

// Move the element at index up until heap property restored
void heapifyUp(int index) {
while (index > 0) {
int parent = (index - 1) / 2;
if (isHigherPriority(data_[index], data_[parent])) {
std::swap(data_[index], data_[parent]);
index = parent;
} else {
break;
}
}
}

// Move the element at index down until heap property restored
```

```
void heapifyDown(int index) {
int size = static_cast<int>(data_.size());
while (true) {
int left = 2 * index + 1;
int right = 2 * index + 2;
int largest = index;

if (left < size && isHigherPriority(data_[left], data_[largest])) {
largest = left;
}
if (right < size && isHigherPriority(data_[right], data_[largest])) {
largest = right;
}
if (largest != index) {
std::swap(data_[index], data_[largest]);
index = largest;
} else {
break;
}
}
}
};

/* ---------- Helper parsing functions ---------- */

int priorityForCategory(const std::string &category) {
if (category == "Boss") return 5;
if (category == "Subordinate") return 4;
if (category == "Peer") return 3;
if (category == "ImportantPerson") return 2;
if (category == "OtherPerson") return 1;
return 0; // unknown category, lowest priority
}

long dateToValue(const std::string &date) {
int mm = 0, dd = 0, yyyy = 0;
char d1 = 0, d2 = 0;
std::istringstream iss(date);
iss >> mm >> d1 >> dd >> d2 >> yyyy;
return static_cast<long>(yyyy) * 10000L +
static_cast<long>(mm) * 100L +
static_cast<long>(dd);
}
```

```
// Trim leading/trailing whitespace of a string
std::string trim(const std::string &s) {
const char *ws = " \t\r\n\f\v";
std::string::size_type start = s.find_first_not_of(ws);
if (start == std::string::npos) return "";
std::string::size_type end = s.find_last_not_of(ws);
return s.substr(start, end - start + 1);
}

// Parse "<category>,<subject>,<date>" (the text AFTER "EMAIL ")
// into an Email. Fields are comma-delimited; the subject may contain
// spaces (and even commas, since the LAST comma splits off the date).
bool parseEmailLine(const std::string &rest, Email &e) {
std::string::size_type firstComma = rest.find(',');
if (firstComma == std::string::npos) return false;

std::string::size_type lastComma = rest.rfind(',');
if (lastComma == firstComma) return false; // need two commas

std::string category = trim(rest.substr(0, firstComma));
std::string subject = trim(rest.substr(firstComma + 1, lastComma - firstComma - 1));
std::string date = trim(rest.substr(lastComma + 1));

e.sender = category;
e.subject = subject;
e.date = date;
e.priority = priorityForCategory(e.sender);
e.dateValue = dateToValue(e.date);

return true;
}

/* ---------------- Main program ---------------- */

int main(int argc, char *argv[]) {
std::ifstream file;
std::istream *in = &std::cin;

if (argc > 1) {
file.open(argv[1]);
if (!file) {
```

```
std::cout << "Could not open file: " << argv[1] << "\n";
return 1;
}
in = &file;
}

MaxHeap heap;
std::string line;

while (std::getline(*in, line)) {
line = trim(line);

if (line.empty()) {
continue; // skip blank lines
}

if (line.compare(0, 6, "EMAIL ") == 0) {
Email e;
if (parseEmailLine(line.substr(6), e)) {
heap.insert(e);
}
}
else if (line == "NEXT") {
Email top;
if (heap.peekMax(top)) {
std::cout << "Next email:\n"
<< "Sender: " << top.sender << "\n"
<< "Subject: " << top.subject << "\n"
<< "Date: " << top.date << "\n";
} else {
std::cout << "No emails to read.\n";
}
}
else if (line == "READ") {
Email discarded;
// Remove the top email (if any) without displaying it.
heap.extractMax(discarded);
}
else if (line == "COUNT") {
std::cout << "There are " << heap.count() << " emails to read.\n";
}
// Unknown commands are silently ignored
}
```

```
return 0;
}
```

e. Claude has the same output as copilot, and the same issue of heap overflow. It has the same time complexity of O(log n) and the same space complexity of O(n). The code has good comments and clear function names.

f. I felt as though I could have picked either program as both had no problems and completed the task successfully. However, I picked Copilot as it accomplished the task in less lines of code.

g. I added comments throughout Copilots code, making it more readable and easier to understand. I could not find anything else to be improved.